



Major Project Report

August - December 2024

Autonomous Drone Delivery System



Sathvika Balasubramanian (21101004)

Kesia Ajith (21101027)

Mehak Panwar (21101026)

Kanwalpreet Kaur (21101030)

Under the Guidance of
Professor Prabhjeet Singh

Department of Aerospace Engineering
Punjab Engineering College (Deemed To be University)

DECLARATION

We hereby declare that the minor project work titled “*Autonomous Drone Delivery System*” is our own work carried out at *Our Institute* as part of the major project for the *B.Tech. Aerospace Engineering*, at *Punjab Engineering College*. This work was done under the guidance of *Dr. Prabhjeet Singh*.

Sathvika Balasubramanian (21101004)

Kesia Ajith(21101027)

Mehak Panwar(21101026)

Kanwalpreet Kaur(21101030)

Date: _____

Certified by:

Dr. Prabhjeet Singh
Assistant Professor
Department of Aerospace Engineering
Punjab Engineering College, Chandigarh
(Deemed to be University)
Chandigarh – 160012, India

Acknowledgement

Our sincere gratitude to our mentors, Dr.Prabhjeet Singh for their constant support throughout the whole project. We express our sincere gratitude to the Aerospace Department for providing us with the Pixhawk Flight Controller for the project. We would also like to thank Mr. Reeshav Chowdhury for his support in providing us with the materials and components.

Abstract

The transportation sector has undergone phenomenal transformations since the introduction of Unmanned Aerial Vehicles (UAVs). The capabilities of UAVs have extended across various domains, ranging from television broadcasting to door-to-door delivery services. The use of autonomous UAVs for last-mile deliveries has the potential to significantly enhance the e-commerce industry by improving both performance and efficiency. The objective of this research is to develop an autonomous drone capable of picking up small payloads and delivering them to specified destination locations. This study involves the creation of a simulation model using the Robot Operating System (ROS) and its subsequent implementation and testing on a pixhawk-powered F450-sized quadcopter platform.

List of Figures

2.1	System architecture of the autonomous delivery drone.	8
3.1	Layout of the arena traversed by the drone	12
3.2	CAD Design of the quadcopter	14
3.3	Flowchart depicting drone sizing method	14
3.4	Propeller spin configuration	16
3.5	Connections between various components. Image Credit: Ardupilot.org .	18
3.6	Pixhawk and Raspberry Pi integration	20
3.7	Final integration of the quadcopter	22
3.8	Image processing using Raspberry Pi	25
3.9	ArUco Marker	26
3.10	QGroundControl interface on ROS2	29
3.11	Gazebo on ROS2	29
3.12	The Raspberry Pi 4	30
3.13	ROS2	31
3.14	PX4 and QGroundControl Setup	32
4.1	Screenshot for gazebo environment and sensor monitoring	35
4.2	Takeoff Command sent from QGroundControl	35
4.3	Takeoff performed by drone	36
4.4	Proceeding towards drop zone	36
4.5	Arrival at drop zone	37
4.6	Proceeding towards drop zone	37
4.7	Landing back at the Aruco Marker	38
5.1	Drone built for autonomous delivery	41

Chapter 1

Introduction

In recent years, the logistics and delivery industry has undergone a profound transformation due to the increasing reliance on e-commerce and the need for swift and efficient delivery mechanisms. Autonomous drones have emerged as a disruptive technology capable of addressing the challenges associated with traditional last-mile delivery, including traffic congestion, high operational costs, and environmental concerns. By bypassing ground-based obstacles and leveraging the flexibility of aerial routes, drones offer a novel solution to the ever-growing demands for urban logistics and delivery efficiency.

The primary objective of this project is to implement an autonomous drone delivery system using the F450 quadcopter platform integrated with a Pixhawk flight controller and ROS (Robot Operating System). The drone is equipped with GPS for autonomous navigation and is designed to pick up and deliver payloads while passing through predefined waypoints. QGroundControl is utilized for mission planning and monitoring, ensuring efficient and reliable operations. This project not only demonstrates the feasibility of autonomous delivery but also contributes to the broader understanding of integrating drones into real-world logistics networks.

1.1 Motivation

The motivation for this study stems from the potential of autonomous drones to revolutionize delivery systems by providing an efficient, scalable, and sustainable alternative. The integration of drones into the logistics chain not only enhances delivery times but also reduces the carbon footprint associated with ground-based transportation. As cities grow denser and infrastructure becomes more complex, the need for autonomous systems capable of precise navigation and payload delivery has become increasingly apparent.

1.2 Need for Study

The need for this study is further underscored by the limitations of existing delivery methods. Ground-based vehicles are often constrained by traffic conditions, high costs, and limited reach in remote or inaccessible areas. By contrast, drones offer a cost-effective, agile, and environmentally friendly alternative. However, the successful implementation of such systems requires addressing key challenges such as navigation, control, payload stability, and mission reliability. This project aims to bridge these gaps by developing a comprehensive system that integrates cutting-edge technologies to ensure robust performance.

Timeline of the Project

Upto Mid-sem (August- September)

- **Literature Review and Concept Development** : Define objectives, payload specs, and operational requirements.
- **System Design** : Design CAD.
- **Prototype Development** : Procurement of hardware components

Post Mid-sem (October - December)

- **Software & System Integration**
- **Model Testing**
- **Field Trials**

Chapter 2

Literature Review

2.1 Paper: The Urban Last Mile Problem: Autonomous Drone Delivery to Your Balcony

Authors: Gino Brunner, Bence Szebedy, Simon Tanner, and Roger Wattenhofer, 2019

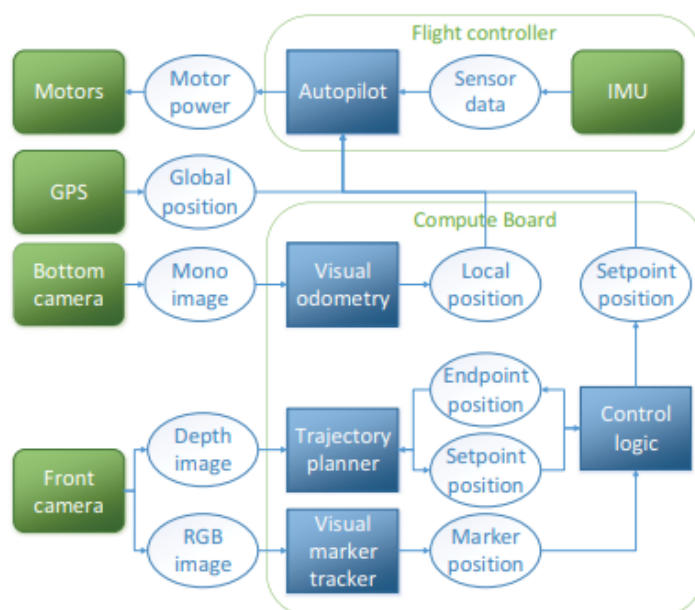


Figure 2.1: System architecture of the autonomous delivery drone.

This paper focuses on utilizing Pixhawk to address the challenges of **urban last-mile delivery**, emphasizing dense environments where precise navigation is critical. Unlike conventional rural delivery approaches, this system leverages visual marker navigation using WhyCon markers to identify specific drop-off points such as balconies or porches. It combines high-altitude GPS navigation with visual odometry for precise low-altitude operations, ensuring reliable localization in GPS-compromised urban settings. A unique feature is its trajectory planner, which uses a 3D occupancy grid and B-spline trajec-

ries for smooth, obstacle-free navigation, supported by collision-avoidance mechanisms. The system architecture (Fig1) integrates key components for autonomous delivery. The Pixhawk-based flight controller runs the PX4 autopilot for stabilization and navigation, while a companion compute board executes advanced tasks like visual odometry, marker tracking, and trajectory planning using the Robot Operating System (ROS). Hardware includes downward-facing cameras for odometry, front-facing RGB, and depth cameras for marker detection and collision avoidance. Communication between the components is managed via the MAVLink protocol and MAVROS, enabling real-time updates and control. This modular and scalable design ensures adaptability for urban delivery missions, balancing precision and efficiency.

2.2 Paper: Aerial Delivery System

Authors: *Ishwari Deshpande, Abhishek Patil, and Prof. Vinaya Tapkir, 2024*

The paper "Aerial Delivery System" by Ishwari Deshpande, Abhishek Patil, and Prof. Vinaya Tapkir presents a drone prototype designed for autonomous emergency deliveries, particularly in inaccessible areas. The system, based on a quadcopter platform, incorporates a Pixhawk 2.4.8 flight controller, GPS, servo-based pick-and-drop functionality, and modular hardware for customization. Key features include autonomous navigation, return-to-home capability, and payload handling up to 372 grams. The study uses ANSYS for stress analysis and optimization, ensuring stability and performance under load, achieving a thrust-to-weight ratio of 2:1. Battery endurance is calculated at approximately 7.5 minutes, making the system suitable for short-range operations like disaster relief and medical supply delivery. Future improvements focus on enhancing reliability, extending flight range, and increasing payload capacity, demonstrating the potential of UAVs in addressing logistical and humanitarian challenges.

2.3 Paper: Developing a Prototype for Autonomous Drones

Authors: *Shreya Kalta, Harsh Chauhan, Guladab Bawa, Abhishek Bag, Gunmeet Singh*

This paper explores the design and development of an autonomous drone system emphasizing efficiency, safety, and reliability. The proposed prototype integrates advanced technologies, including obstacle avoidance and weather monitoring sensors, supported by machine learning algorithms for real-time decision-making. Key features include GPS navigation, a stable flight control system, and wireless communication. The study highlights the autonomous system's ability to perform complex missions and proposes advancements like solar-powered flight and enhanced algorithms for future improvements.

This research demonstrates the potential applications of autonomous drones in industrial, logistics, and security sectors while addressing safety and privacy concerns.

2.4 Paper: Secure Skies: Developing a Remotely Piloted Aerial System with Defense M

Authors: *Md. Tauseef, Thanushree M P, K Sravani, Lohith R, T Sai Sankalp Reddy*

This paper presents the development of a Remotely Piloted Aerial System (RPAS) with integrated defense mechanisms, designed to enhance operational security and efficiency. Key components include advanced sensors (GPS, cameras) for real-time monitoring and targeting, and a web-based interface enabling intuitive remote control. A SpeedyBee F405 WING controller is employed for precise control of the defense mechanisms, ensuring reliable actuation of non-lethal projectiles such as rubber bullets or pepper spray. This modular system supports mission-specific payloads, improving versatility and adaptability. The RPAS also features an optimized CAD-designed airframe for aerodynamics and stability, and brushless motors for efficient propulsion. Testing demonstrated the system's effectiveness in surveillance, reconnaissance, and precision-targeted interventions. Future work involves integrating AI-driven target detection models, swarm technology for collaborative missions, and energy innovations to extend flight time. This research highlights RPAS's transformative applications in military, law enforcement, and disaster response contexts, emphasizing safety and adaptability.

2.5 Paper: Autonomous Quadcopter Delivery System

Authors: *Hasan Mudhafar Qays, Aymen Dawood Salman, Bassim Abdul Baki Jum*

This paper outlines the design and implementation of an autonomous delivery system using a quadcopter. Central to the system is the Pixhawk flight controller, a versatile, open-hardware autopilot platform supporting Ardupilot and PX4 flight stacks. Pixhawk enables high-precision control with integrated sensors such as a gyroscope, accelerometer, magnetometer, and barometer, making it ideal for UAV autonomy. The system leverages Pixhawk's compatibility with MAVLink protocol for seamless communication with a Raspberry Pi 3, which acts as an onboard processing unit. Key features include dynamic path planning via the Next Nearest Location First (NNLF) algorithm, power estimation for efficient energy use, and user notifications through email. A Google Maps-based web interface allows users to input delivery requests, with paths recalculated dynamically if new destinations are added. This robust integration highlights Pixhawk's role in enhanc-

ing UAV navigation and mission management for applications ranging from delivery to surveillance in hazardous areas.

Chapter 3

Theory

3.1 Mission Details

3.1.1 Objective

To lift and hover a payload of 250 grams at an altitude of 10 meters above ground. The drone must be capable of traversing through the specified path independently maintaining steady velocity and stability and be able to drop it at the designated location.

Given below is a layout of the arena traversed by the drone



Figure 3.1: Layout of the arena traversed by the drone

3.2 Drone Configuration

A quadcopter configuration is selected for the current project with a payload limit of 250 grams. The sizing of a drone is an iterative process that depends on the payload capacity and the material of the frame. Given below is the CAD model of the quadcopter.

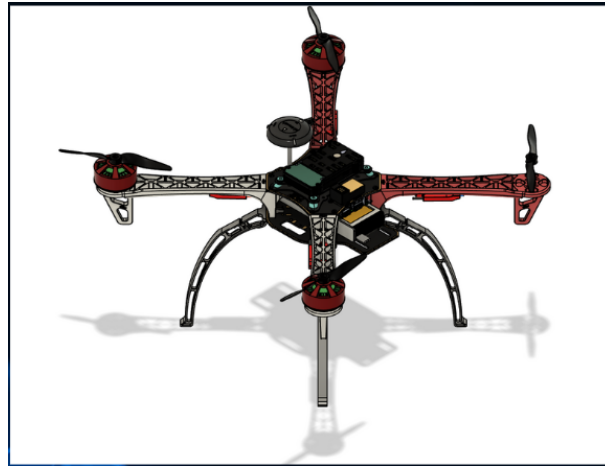




Figure 3.2: CAD Design of the quadcopter

3.2.1 Drone Sizing

The design and configuration of a quadcopter involve the following key steps:

Payload and Thrust Calculation

- **Payload weight (W_p):** The weight the quadcopter must carry, including sensors, cameras, and other equipment.
- **Total weight (W_t):** The combined weight of the payload, frame, motors, battery, ESCs (Electronic Speed Controllers), and wiring, including a safety margin (typically 20-30% extra).
- **Thrust-to-weight ratio (R_t):** A ratio of at least 2:1 is required for stable flight, with 3:1 preferred for agility and additional safety.
- **Thrust per motor (T_m):**

$$T_m = \frac{W_t \cdot R_t}{4}$$

where W_t is the total weight, R_t is the thrust-to-weight ratio, and 4 represents the number of motors.

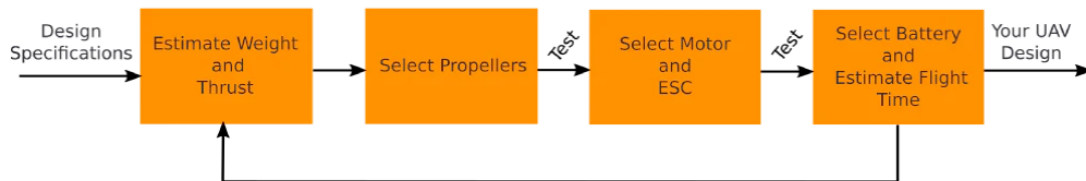


Figure 3.3: Flowchart depicting drone sizing method

Propeller Selection

- **Propeller size:** Larger propellers (e.g., 10-12 inches) provide more thrust at lower RPM but require more torque, while smaller propellers (5-7 inches) operate efficiently at higher RPMs.
- **Material:** Common options include lightweight plastic or durable carbon fiber.
- **Pitch:** Higher pitch increases thrust at high speeds but reduces efficiency.

Motor Selection

- **KV rating:** Defines the motor's speed in RPM per volt. Low KV motors (e.g., 800-1200 KV) suit larger propellers, while high KV motors (e.g., 2000-2500 KV) are suitable for smaller propellers.
- **Torque:** The motor must generate sufficient torque to spin the selected propellers at the desired thrust level.

Battery Sizing

- **Battery capacity (C_b):** Determines flight time, measured in mAh. Higher capacity increases flight time but adds weight.
- **Voltage (V_b):** Matches motor and ESC requirements. Typical options include 3S (11.1 V) or 4S (14.8 V) LiPo batteries.
- **Discharge rate (C_r):**

$$C_r = \frac{I_d}{C_b}$$

where I_d is the current draw, calculated as:

$$I_d = \frac{T}{\eta} \cdot 4$$

Here, T is the thrust, and η is motor efficiency.

Frame Dimensions

- **Motor-to-motor distance:** Determined by the size of the propellers:
 - For 5-7 inch propellers: 250-350 mm frame size.
 - For 10-12 inch propellers: 450-600 mm frame size.
- **Material:** Carbon fiber is lightweight and strong, ideal for heavy payloads.

Electronics Selection

- **ESCs (Electronic Speed Controllers):** Must handle the current draw of the motors at full throttle, with a safety margin of 20%.
- **Flight controller:** Options depend on the application:
 - Pixhawk for advanced control.
 - Betaflight for acrobatic flight.
- **Sensors:** Gyroscopes, accelerometers, GPS, barometers, and cameras can be added as needed.

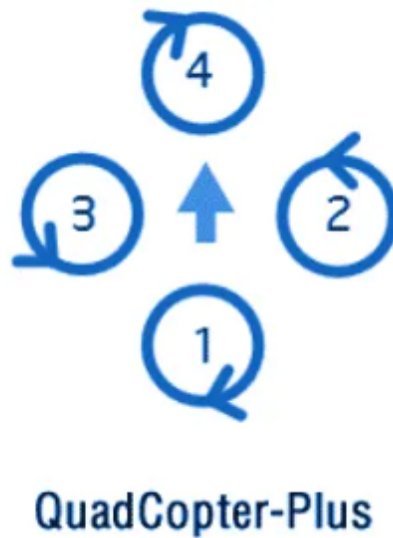


Figure 3.4: Propeller spin configuration

Weight Optimization

- Use lightweight materials for the frame and payload mounts.
- Minimize wiring length and avoid oversized components to reduce unnecessary weight.

Testing and Fine-Tuning

- Assemble the quadcopter and test for hover stability.
- Balance the frame to ensure proper thrust distribution.
- Optimize flight parameters using the flight controller's software.

3.3 Drone Electronics

A quadcopter drone consists of a frame supporting four motors, each driving a propeller. The motors are connected to Electronic Speed Controllers (ESCs) that regulate their speed based on signals from the flight controller. The flight controller, the drone's brain, processes input from sensors (gyroscope, accelerometer, GPS) and commands from the transmitter, received via a receiver module. The battery powers all components, with power distributed through a Power Module or integrated into the flight controller depending on the type of flight controller. Propellers are mounted on motors, two rotating clockwise and two counterclockwise, To balance thrust and counteract torque, preventing the drone from spinning uncontrollably. All components are interconnected with wires, ensuring coordinated functionality.

Here is a list of the specifications of the components used in the project:

- FlySky FS-i6 2.4G 6CH PPM RC Transmitter With FS-IA6B Receiver
- 35A 2-5S BLHELIS 4 in 1 ESC
- Orange 11.1V 2200mAh 30C 3S Lithium Polymer Battery Pack
- Pixhawk 2.4.8 Basic Flight Controller kit with GPS Module Combo Kit
- 915Mhz 100mW Mini Telemetry Kit
- INVENTO 1600KV BLDC Brushless Motor + 6045 6x4.5 6 inch Propeller Quadcopter Mini Kit ISC 1558-2B
- F450 / Q450 Quadcopter Frame(PCB Version with Integrated PCB) + Plastic Landing Gear Combo Kit – Made in INDIA
- Raspberry Pi 4 Model-B with 1 GB RAM
- OV5647 5MP 1080P IR-Cut Camera for Raspberry Pi 3/4 with Manual Day Night Mode

nents.

Switch: Acts as a safety feature to turn the power on or off from the battery to Pixhawk. It is positioned between the power module and Pixhawk.

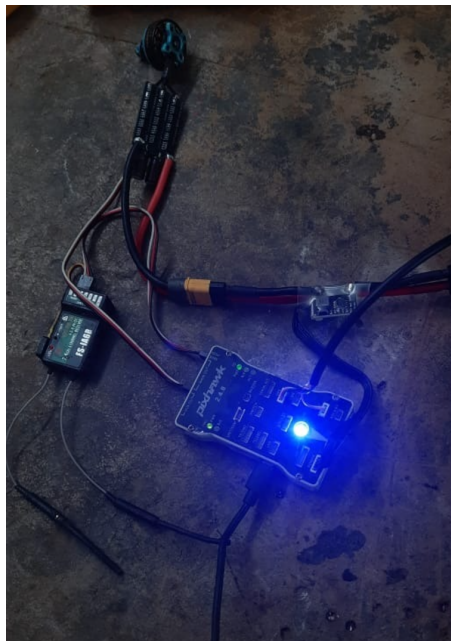
Telemetry Module: Facilitates two-way communication between Pixhawk and a ground control station (tablet or PC). It is connected via the TELEM port on Pixhawk, allowing remote monitoring and control.

GPS: Provides location, altitude, and orientation data for navigation. It is connected to the dedicated GPS port on Pixhawk, enabling precise positioning.

Receiver: Receives signals from the transmitter and forwards them to Pixhawk through the RC IN port. It simplifies the connection by combining multiple channel signals into one.

Gimbal: Stabilizes the camera for smooth video and image capture. It connects to Pixhawk via an auxiliary output, allowing camera angle adjustments during flight.

Transmitter: Sends control inputs to the receiver, which passes them to Pixhawk. The transmitter controls the drone's movements and functions via sticks and switches.



Images: The images of connections during component testing and hardware integration are as follows:

Camera and Raspberry Pi Integration for Payload Drop Operations

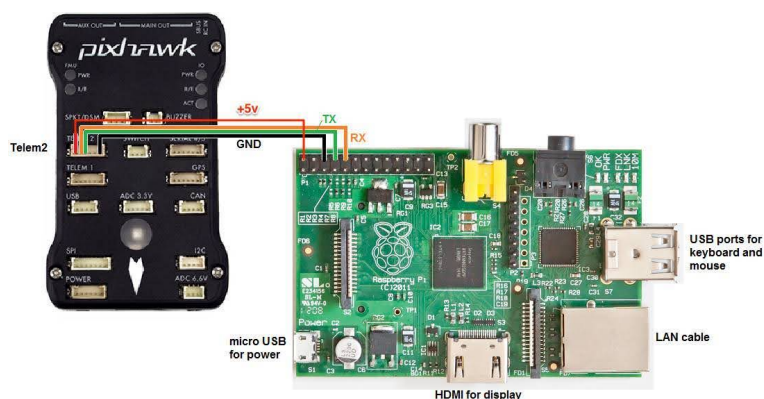


Figure 3.6: Pixhawk and Raspberry Pi integration

Camera Usage: The camera is utilized for identifying ArUco markers to facilitate payload drop operations. A Raspberry Pi has been integrated into the system to handle the camera's functionality. Below is an overview of how the Raspberry Pi operates in conjunction with the Pixhawk flight controller for this purpose.

- **Connection Overview** The Pixhawk and Raspberry Pi are connected via the TELEM2 port on the Pixhawk and the GPIO pins on the Raspberry Pi. This allows data communication between the two devices, enabling the Raspberry Pi to receive telemetry data and send commands to the Pixhawk.
- **Wiring Details**
 - **5V (Red Wire):** Powers the Raspberry Pi from the Pixhawk's TELEM2 port.
 - **GND (Black Wire):** Provides a common ground connection between both devices for stable communication.
 - **TX (Transmission) (Green Wire):** Pixhawk's TX pin sends data to the Raspberry Pi's RX (Receive) pin.
 - **RX (Reception) (Orange Wire):** Pixhawk's RX pin receives data from the Raspberry Pi's TX (Transmit) pin.
- **Raspberry Pi Power**
 - **Micro USB:** Used to provide additional power to the Raspberry Pi, especially if peripherals like a camera are connected.
- **Camera Connection**

Connect a compatible camera to the Raspberry Pi via the camera interface (CSI)

port or a USB port, depending on the camera type. The Raspberry Pi can then process the video feed and overlay telemetry data from the Pixhawk.

- **Communication Purpose**

The Raspberry Pi processes camera data and combines it with telemetry from the Pixhawk. It can send commands back to the Pixhawk for autonomous missions, or stream the camera feed with telemetry data to a ground station.

- **Usage Scenarios**

- **FPV (First-Person View):** Stream real-time video with telemetry data for enhanced navigation.
- **Autonomous Missions:** Use image processing for tasks like object detection or waypoint navigation.

- **Usage Scenarios**

- **FPV (First-Person View):** Stream real-time video with telemetry data for enhanced navigation.
- **Autonomous Missions:** Use image processing for tasks like object detection or waypoint navigation.

Summary: A quadcopter drone is a system comprising interconnected components, including motors, ESCs, a flight controller, sensors, a receiver, a transmitter, and a battery. These elements work together to ensure stable flight and precise control. The motors are regulated by a 4-in-1 ESC based on signals from the Pixhawk flight controller, which processes input from sensors and the receiver.



Figure 3.7: Final integration of the quadcopter

A Raspberry Pi is integrated to handle camera functions for identifying ArUco markers in payload drop missions, communicating with the Pixhawk through the TELEM2 port.

This setup allows the Raspberry Pi to process video feeds, overlay telemetry data, and execute autonomous tasks like object detection and waypoint navigation, making the drone versatile for various applications.

3.4 Payload Drop Mechanism

The Raspberry Pi processes live video from an onboard camera to detect an ArUco marker, which marks the designated drop zone. Upon identifying the marker, the Raspberry Pi sends a command to the Pixhawk, triggering a servomotor that releases the payload. The payload drop process using a servomotor can further be explained as follows:

1. Detection of ArUco Marker

- The onboard camera, connected to the Raspberry Pi, captures a live video feed.
- The Raspberry Pi processes this feed using computer vision algorithms to detect the ArUco marker.
- Once the marker is identified and verified as the designated drop zone, the Raspberry Pi calculates the drone's position relative to the marker.

2. Communication Between Raspberry Pi and Pixhawk

- After detecting the ArUco marker, the Raspberry Pi sends a command to the Pixhawk through the TELEM2 port using the MAVLink protocol.
- This command includes precise location data and triggers the payload release mechanism.

3. Servomotor Activation

- The Pixhawk flight controller receives the drop command from the Raspberry Pi.
- The Pixhawk then sends a PWM (Pulse Width Modulation) signal to the SG90 servomotor connected to one of its auxiliary ports.
- The servomotor responds by rotating to a specific angle, releasing the payload by activating a latch or dropping mechanism.

4. Payload Release

- As the servomotor rotates, it disengages the holding mechanism securing the delivery package.

- The package is then released from the drone and falls directly onto the marked drop zone.

5. Post-Drop Procedure

- After the payload is dropped, the servomotor resets to its original position.
- The Raspberry Pi processes this feed using computer vision algorithms to detect the ArUco marker.
- The drone can either return to its home base or continue to the next waypoint based on pre-programmed instructions.

3.5 Ground Control Station

3.5.1 Ground Control Software and ArUco Marker Processing

QGroundControl: Ground Control Software for Autonomous Delivery Drones

Introduction: In the development of our autonomous delivery drone system, selecting appropriate ground control software is critical. Since the Pixhawk flight controller operates on ArduPilot firmware, it was essential to choose a software solution that integrates seamlessly with this platform while offering the functionality required for mission planning, calibration, and autonomous operations.

Role of Ground Control Software: Ground control software serves as the central interface for configuring, controlling, and monitoring drones. It enables users to calibrate sensors, plan missions, and execute autonomous operations. Among the popular options are QGroundControl and Mission Planner, both of which offer unique features tailored to different drone systems and use cases.

Why QGroundControl Was Chosen: For our autonomous delivery drone system built on a Pixhawk flight controller, we selected QGroundControl after evaluating several options. The reasons for this choice include:

- **Comprehensive ArduPilot Integration:** QGroundControl is designed for compatibility with ArduPilot, ensuring seamless operation with the Pixhawk flight controller.
- **User-Friendly Mission Planning Tools:** The software allows for easy creation of waypoint-based delivery routes, defining actions such as package drops, and configuring automated behaviors like Return-to-Launch (RTL).

- **Intuitive Calibration and Configuration:** It provides straightforward workflows for calibrating components like accelerometers, compasses, radios, and ESCs, making it accessible for users of all skill levels.
- **Robust Telemetry Support:** Real-time data visualization during missions ensures efficient monitoring and troubleshooting, allowing for adjustments on the fly.
- **Extensive Parameter Customization:** The software offers granular control over drone parameters, enabling fine-tuning of flight dynamics and mission optimization.

Configuration Steps: Using QGroundControl, we configured and managed the autonomous delivery drone system through several essential steps:

1. Updated the firmware on the Pixhawk controller to ensure compatibility with ArduPilot.
2. Calibrated the accelerometer, compass, and radio to guarantee accurate sensor readings and input mapping.
3. Synchronized ESCs with throttle controls to ensure precise motor performance.

3.5.2 Image Processing of ArUco Markers Using Raspberry Pi

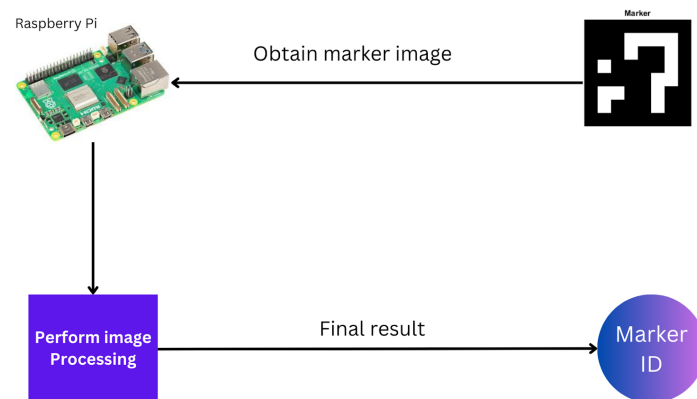


Figure 3.8: Image processing using Raspberry Pi

Hardware Requirements: To implement image processing on a Raspberry Pi, the following components are required:

- **Raspberry Pi Board:** For example, a Raspberry Pi 4.
- **Camera Module:** For example, the Raspberry Pi Camera V2.

Software Requirements: The OpenCV library must be installed on the Raspberry Pi to facilitate image processing tasks.

ArUco Marker Detection Process: The following steps outline the method for detecting and decoding ArUco markers using a Raspberry Pi:

1. **Extract the ArUco Marker from the Image:**

- Capture an image using the camera module.
- Use OpenCV to detect the square marker by identifying its black border.
- Isolate the marker from the image for further analysis.

2. **Remove Extra Padding:**

- Crop the image to remove unnecessary surrounding pixels or padding around the marker.
- Retain only the internal grid of the marker.

3. **Convert the Marker to Binary Format:**

- Divide the cropped marker into a 5x5 grid.
- Focus on the second and fourth columns, which contain the data bits.
- Analyze cells column-wise from top to bottom.

4. **Assign Binary Values:**

- Assign a value of 1 for white cells and 0 for black cells.

5. **Convert Binary to Decimal:**

- Convert the binary sequence from the selected columns into its decimal equivalent to obtain the marker's unique ID.

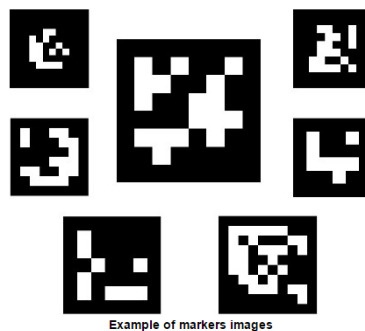


Figure 3.9: ArUco Marker

ArUco Marker Description: ArUco markers are square, binary fiducial markers widely used for camera pose estimation due to their simplicity, speed, and robust detection capabilities. Each marker consists of:

- A thick black border for quick and reliable detection.
- An inner binary matrix that encodes a unique identifier (ID).

For example, a 4x4 marker contains 16 bits. The first, third, and fifth columns serve as parity bits, while the second and fourth columns contain the data bits.

Summary: This systematic approach ensures accurate detection and decoding of ArUco markers in a resource-efficient manner on the Raspberry Pi platform. The combination of QGroundControl and Raspberry Pi-based image processing enhances the autonomous delivery drone system by enabling seamless mission planning, real-time monitoring, and robust payload drop operations.

Objective: To use ROS2 Software in the loop simulation to integrate PX4(PixHawk Firmware), QGroundControl and Raspberry Pi 4 for drone delivery.

Reason:Using ROS2 Software-in-the-Loop (SITL) simulation to integrate PX4 (PixHawk firmware), QGroundControl, and a Raspberry Pi 4 for drone delivery provides several benefits before moving to actual hardware testing:

- **Safety:** Allows you to test flight logic, navigation, and control algorithms without risking damage to the drone or its environment.
- **Cost Efficiency:** Identifies and resolves bugs or issues early, reducing the likelihood of costly hardware repairs or replacements.
- **Iterative Development:** Facilitates rapid prototyping and iterative refinement of code and parameters without needing physical flights.
- **Realistic Testing:** Simulates real-world conditions, including sensor data, environmental factors (e.g., wind), and mission scenarios, ensuring your system can handle diverse challenges.
- **Seamless Integration:** Verifies communication and data flow between PX4, QGroundControl, and the Raspberry Pi 4, ensuring that the components work together as expected.
- **Debugging and Visualization:** Offers tools to visualize flight paths, analyze telemetry, and debug system behavior in a controlled environment.

- **Custom Scenarios:** Enables you to create complex delivery scenarios (e.g., obstacle avoidance, route optimization) that might be impractical to replicate in real-world tests.
- **Time Savings:** Avoids delays caused by weather or other external factors that can interrupt physical testing.

By using ROS2 SITL, you can develop a robust and reliable system, minimizing risks and ensuring smooth deployment in real-world drone delivery operations.

Integrating PX4, QGroundControl, Gazebo, and Raspberry Pi 4 in a ROS2 Software-In-The-Loop (SITL) simulation involves creating a robust ecosystem to design, test, and validate drone operations. This integration streamlines the development of autonomous systems by offering a modular and versatile framework. Each component plays a crucial role in achieving a realistic and efficient simulation environment, ensuring compatibility between the software and hardware layers while enabling advanced functionality.

- **PX4** serves as the firmware foundation, providing the essential flight control logic for drones. It includes features such as sensor fusion, attitude estimation, and advanced flight modes, making it an ideal choice for developing autonomous aerial systems. By running PX4 in SITL mode, the same firmware code used in actual hardware is executed in a simulated environment. This ensures that any mission planning, navigation, or control logic can be tested and validated realistically before deploying on physical drones.
- **QGroundControl** complements PX4 by acting as the Ground Control Station (GCS). It provides an intuitive graphical interface for monitoring and controlling the drone, including real-time telemetry, mission planning, and parameter tuning. When integrated with PX4 in SITL, QGroundControl communicates over a network link, allowing seamless testing of flight plans, autonomous missions, and fail-safe behaviors. This interface bridges the gap between user input and the underlying control algorithms, enabling iterative refinements to system configurations.

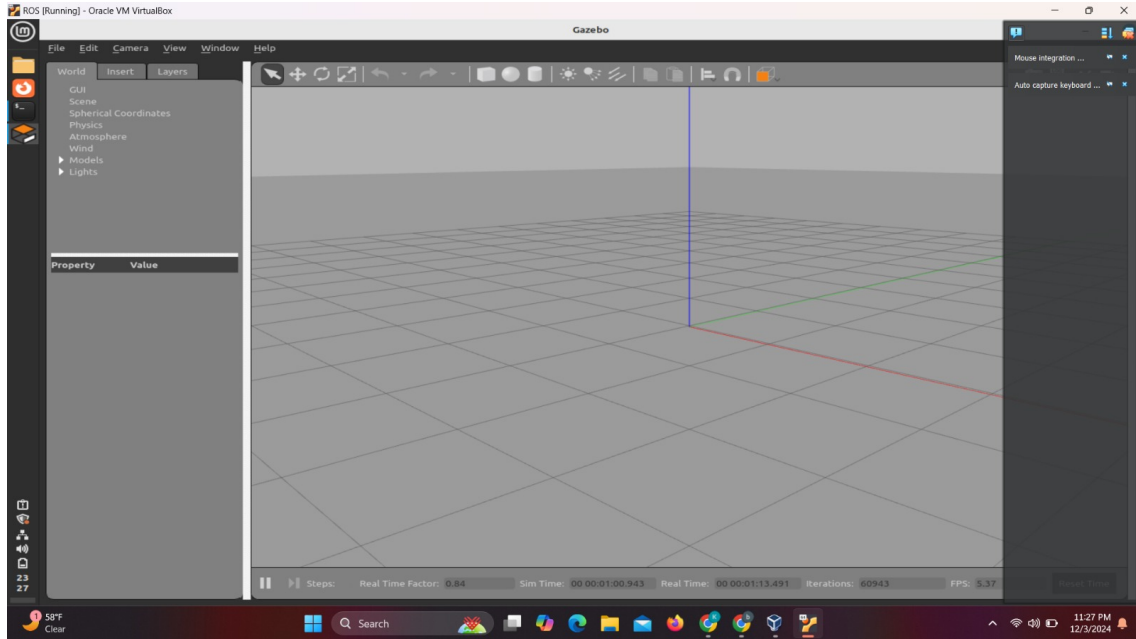


Figure 3.10: QGroundControl interface on ROS2

- **Gazebo**, a high-fidelity simulation environment, integrates tightly with PX4 to provide realistic physics and sensor simulations. It replicates real-world dynamics such as aerodynamics, gravity, and environmental disturbances. Gazebo's ability to simulate various sensors, such as cameras, LiDAR, and GPS, is essential for creating realistic scenarios in drone delivery simulations. For example, integrating Gazebo with PX4 allows testing of obstacle avoidance algorithms or precision landing features without needing physical hardware, ensuring safe and controlled development.

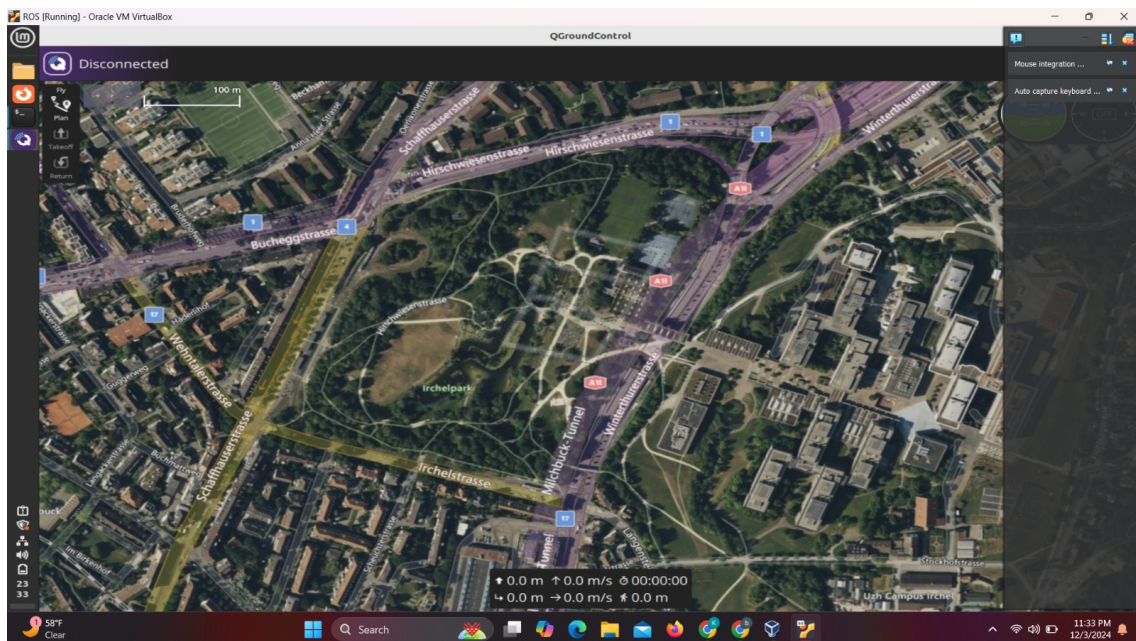


Figure 3.11: Gazebo on ROS2

- The **Raspberry Pi 4** acts as a companion computer in this ecosystem, adding computational power for advanced applications such as computer vision, AI-based decision-making, and data processing. In the SITL framework, the Raspberry Pi can be simulated or directly interfaced with PX4, enabling developers to validate its role in mission execution. Using ROS2, the Raspberry Pi can handle tasks like processing image streams from simulated cameras in Gazebo or sending high-level commands to the PX4 stack, ensuring compatibility and scalability for real-world applications.

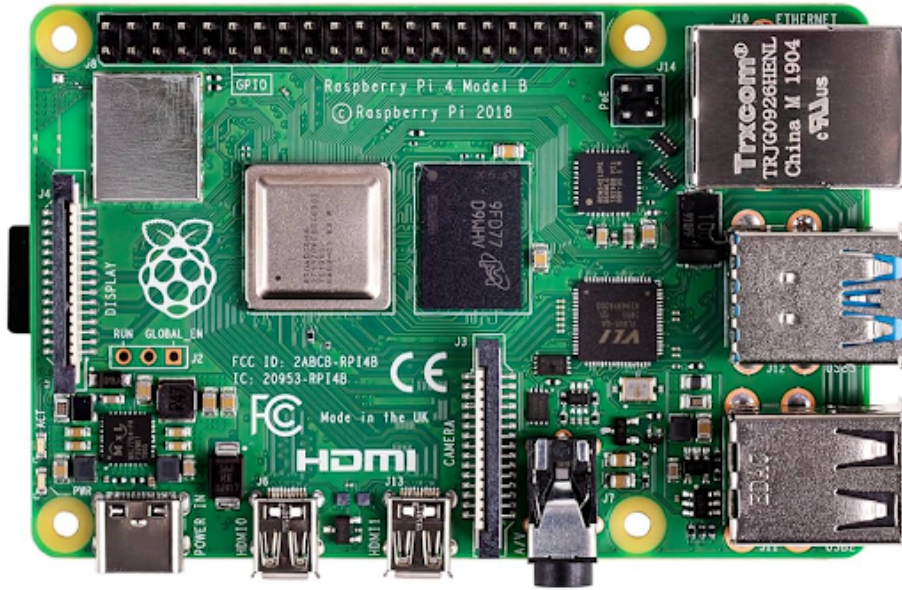


Figure 3.12: The Raspberry Pi 4

- **ROS2** plays a pivotal role in managing communication between all components. With its robust publisher-subscriber model, ROS2 ensures seamless data flow between PX4, Gazebo, and the Raspberry Pi. For instance, sensor data simulated in Gazebo can be published as ROS2 topics, processed by nodes running on the Raspberry Pi, and then translated into control commands for PX4. This modular architecture allows flexibility, making it easy to add or replace components without disrupting the overall system. Overall, the integration of PX4, QGroundControl, Gazebo, and Raspberry Pi 4 in a ROS2 SITL setup creates a comprehensive framework for drone development and testing. It enables developers to simulate complex missions, analyze system performance, and refine algorithms in a risk-free environment. This extensive simulation process ensures that the system is robust, efficient, and ready for deployment in real-world applications, such as drone delivery or autonomous inspections.



Figure 3.13: ROS2

- **OpenCV** in ROS2 serves as a powerful tool for enabling computer vision capabilities in robotic systems. ROS2 (Robot Operating System 2) provides the middleware for communication between different robotic components, while OpenCV offers a comprehensive library for image processing and analysis. Their integration allows for the efficient handling of image streams and enables a variety of vision-based functionalities, from simple image transformations to complex perception tasks.

Integrating PX4, QGroundControl, Gazebo, Raspberry Pi, and a Pi Camera to perform a ROS2-based simulation of a drone delivery system using ArUco markers involves setting up a collaborative ecosystem of tools and platforms. The goal is to simulate and validate the use of ArUco markers for precise navigation, object detection, and landing. This process involves multiple stages, including firmware configuration, simulation environment setup, communication protocols, and vision processing.

- **PX4 Firmware and QGroundControl Setup:** The process begins with configuring PX4, an open-source autopilot firmware that controls the drone's flight behavior. PX4 is configured in Software-In-The-Loop (SITL) mode to allow simulation without physical hardware. The firmware is connected to QGroundControl, which acts as a Ground Control Station (GCS). QGroundControl is used for mission planning, parameter tuning, and real-time monitoring of the simulated drone. It communicates with PX4 via MAVLink, ensuring smooth data exchange between the simulation environment and the GCS.

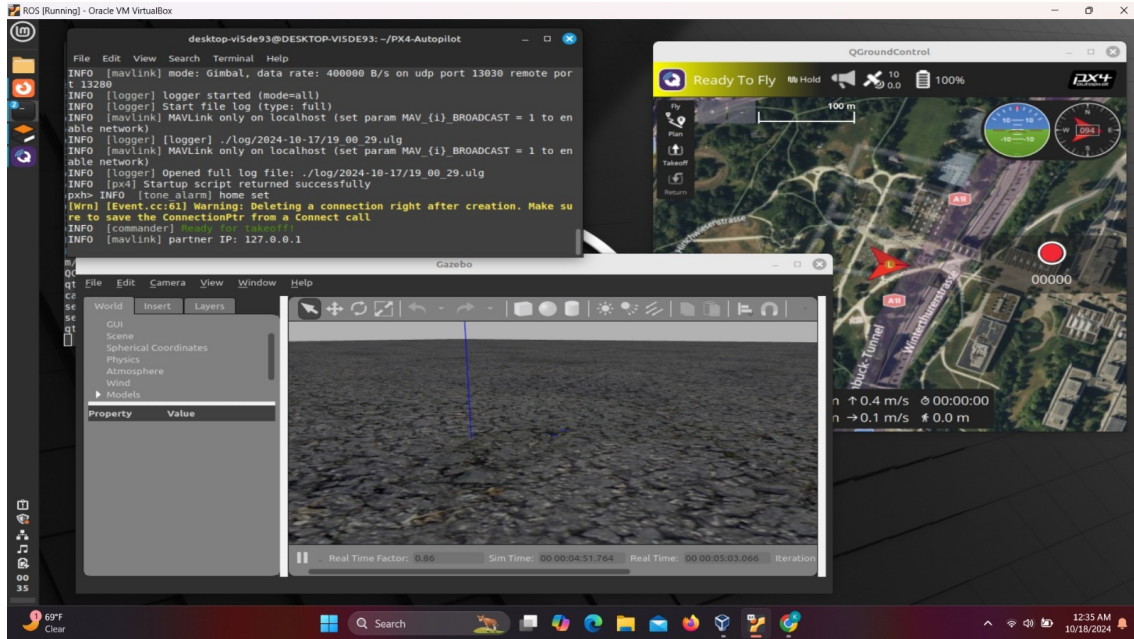


Figure 3.14: PX4 and QGroundControl Setup

- Gazebo Simulation Environment:** Gazebo is utilized as the simulation environment to create a realistic world where the drone can operate. This includes modeling the physics of flight, environmental factors, and the inclusion of ArUco markers as visual landmarks. A simulated drone model compatible with PX4 is added to Gazebo, along with a world environment containing delivery points and ArUco markers for navigation and landing. Gazebo also simulates onboard sensors like cameras, GPS, and IMUs, which are essential for autonomous operations.
- Raspberry Pi and Pi Camera Integration:** The Raspberry Pi, serving as a companion computer, adds computational capabilities to the system. It is responsible for handling high-level tasks such as processing camera feeds and interpreting ArUco marker data. The Pi Camera is integrated into the simulation by emulating its output within Gazebo. Sensor plugins in Gazebo simulate the camera feed, replicating real-world conditions. The simulated camera output is streamed to the Raspberry Pi for processing.
- ROS2 Communication Framework:** ROS2 is used to orchestrate the entire system, enabling seamless communication between components. The camera feed simulated in Gazebo is published as a ROS2 topic, which is subscribed to by a vision-processing node running on the Raspberry Pi. This node utilizes computer vision algorithms to detect ArUco markers in the camera feed, extract their positions, and compute relative coordinates for navigation. ROS2 also handles communication between the Raspberry Pi and PX4, where high-level commands based on marker detection are sent to the autopilot.

- **Mission Workflow:** The simulation workflow begins with a pre-planned mission in QGroundControl, where the drone is tasked with flying to a delivery location. As the drone approaches the delivery point, the Pi Camera, simulated in Gazebo, captures the ArUco markers on the ground. The Raspberry Pi processes the marker data to refine the drone's position and orientation, ensuring precise navigation and landing. This vision-based correction compensates for any GPS inaccuracies, enabling reliable delivery.
- **Testing and Validation:** The final step involves running the complete simulation to test and validate the system. The drone's ability to detect and interpret ArUco markers, adjust its flight path, and land accurately on the designated marker is evaluated. Gazebo provides tools to visualize the simulation, while QGroundControl logs the drone's telemetry and mission status. This iterative process helps refine the system's parameters and logic to ensure robustness before transitioning to real-world testing.

This integrated simulation framework provides a comprehensive platform to develop, test, and optimize drone delivery operations using ArUco markers, ensuring safety and efficiency before deploying on actual hardware.

Chapter 4

Simulations and Results

Achieving precision landing with drones involves a seamless integration of ROS2, PX4, and OpenCV to detect and use ArUco markers for accurate positioning. This method leverages the communication capabilities of ROS2, the reliable flight control system of PX4, and the advanced image processing features of OpenCV to enable drones to autonomously locate and land on predefined markers. The process begins with setting up a simulation environment, processing camera feeds for marker detection, and using the detected data for precision flight adjustments.

The foundation of this system relies on a robust workspace setup. First, the ROS2 environment must be correctly configured to handle drone communication, camera feeds, and marker detection tasks. PX4 is utilized to simulate a drone equipped with a downward-facing camera, enabling it to capture the required visual data during the flight. The ROS GZ bridge is then employed to connect ROS2 with Gazebo, facilitating the transfer of camera image topics and intrinsic data necessary for ArUco detection.

With the workspace prepared, the simulation begins by launching the PX4 SITL environment, which replicates a drone's flight dynamics and onboard systems. A downward-facing camera provides a live feed of the area beneath the drone. This feed is processed in real time using ROS2 nodes equipped with OpenCV. The ArUco tracker node identifies and decodes markers, calculating their position and orientation relative to the camera. This information is critical for the drone to adjust its flight path and align itself precisely over the target marker.

The integration of ROS2's communication framework ensures smooth data flow between various components, including the PX4 flight controller, image processing nodes, and the marker detection system. Once the marker's location is determined, the precision landing node interprets this data and sends commands to the drone's flight controller to execute a controlled descent. The drone continually adjusts its position based on updated

visual feedback until it lands safely and accurately on the target.

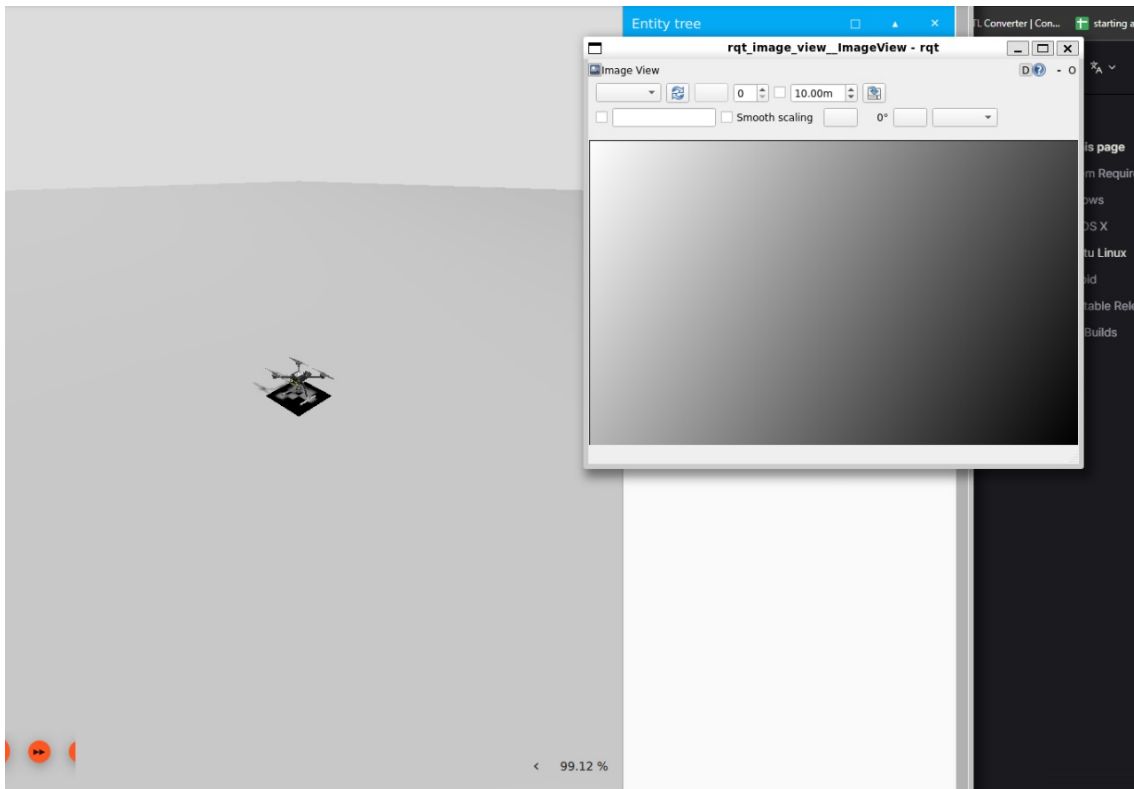


Figure 4.1: Screenshot for gazebo environment and sensor monitoring

4.1 Simulation Working

The simulation begins with a command sent via QGroundControl (QGC) to arm the drone and initiate its flight sequence.

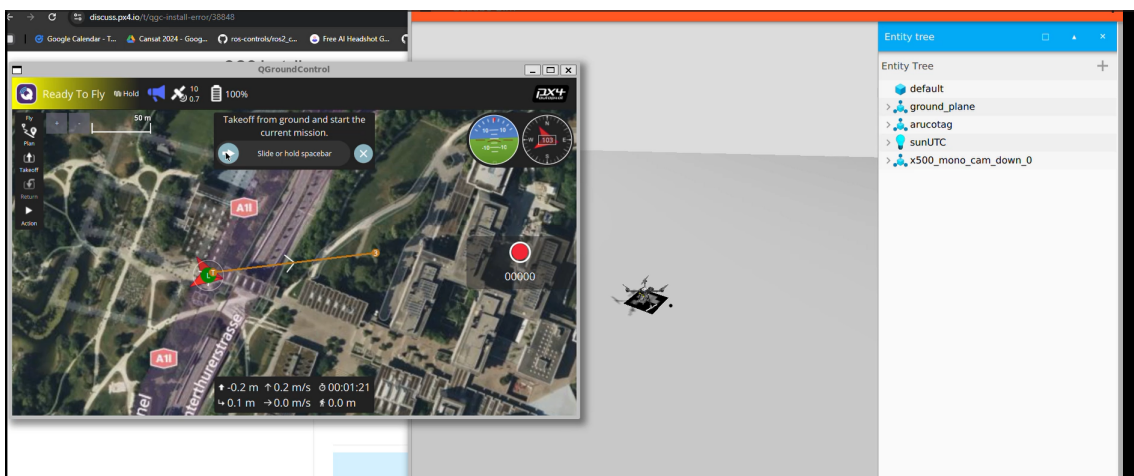


Figure 4.2: Takeoff Command sent from QGroundControl

Once armed, the drone ascends to a predetermined altitude, ensuring it reaches a safe

height above any potential obstacles.

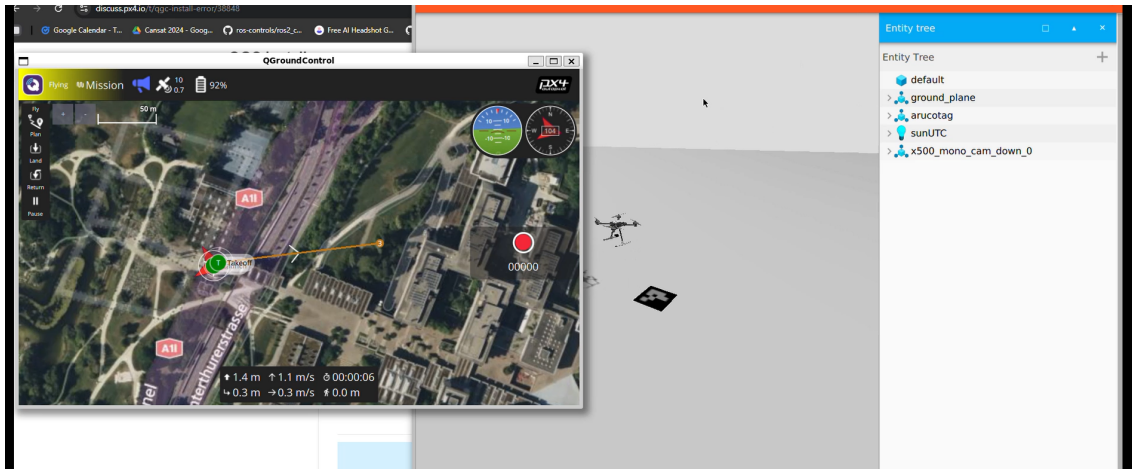


Figure 4.3: Takeoff performed by drone

From this altitude, the drone navigates autonomously to a specified GPS location, leveraging its onboard GPS module and flight control system. This navigation is critical for the drone to reach the payload drop site accurately.

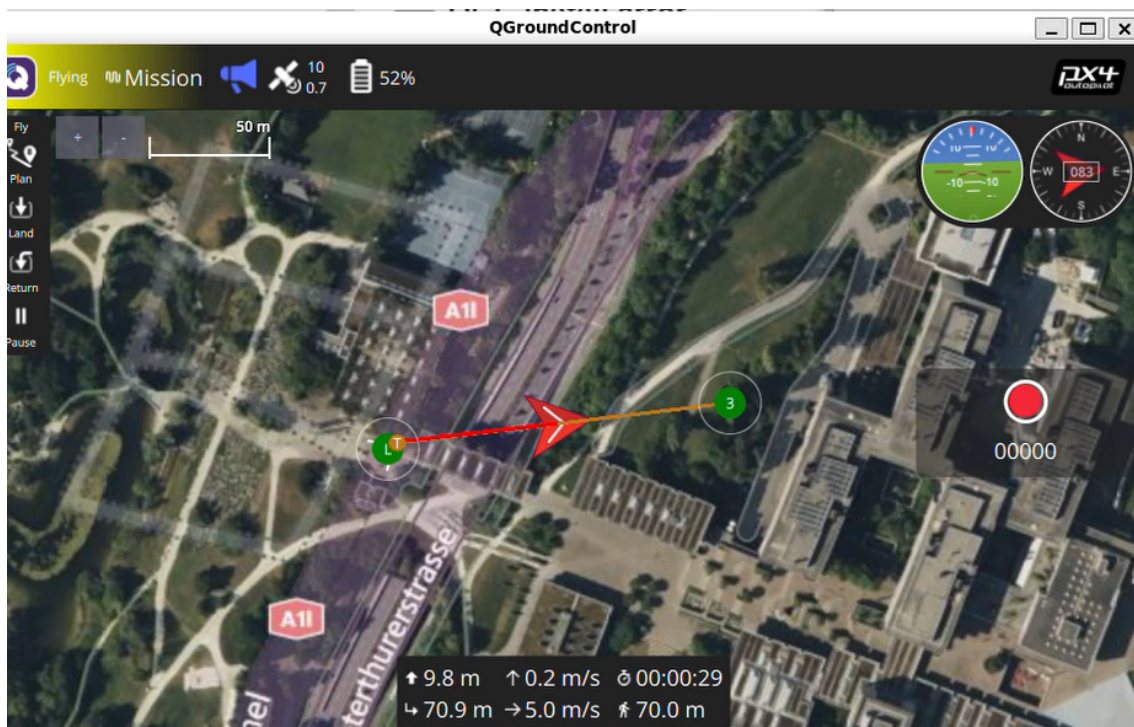


Figure 4.4: Proceeding towards drop zone

Upon arriving at the GPS waypoint, the drone hovers at the designated location and releases its payload. This step may involve using a servo mechanism or a similar device,

depending on the payload delivery system integrated into the drone. The precise execution of this drop ensures that the payload is delivered to the intended target location with minimal deviation.

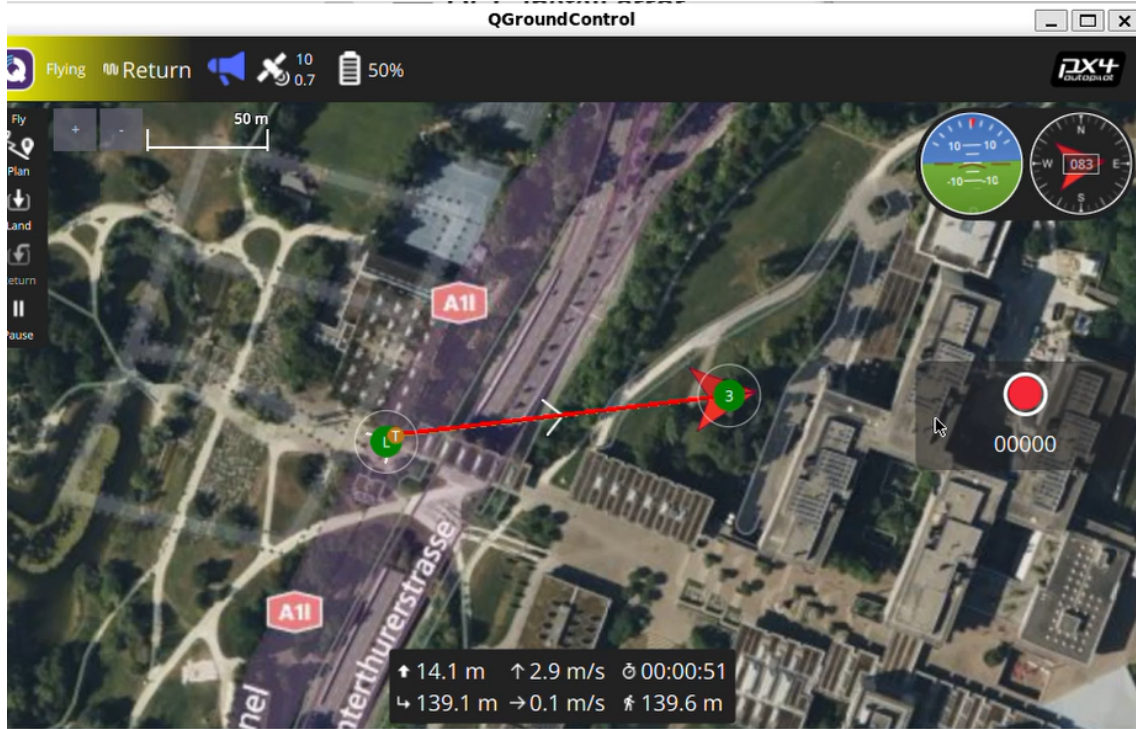


Figure 4.5: Arrival at drop zone

Following the successful delivery, the drone is issued a return-to-home (RTH) command. This command prompts the drone to navigate back to its original takeoff position using GPS data.

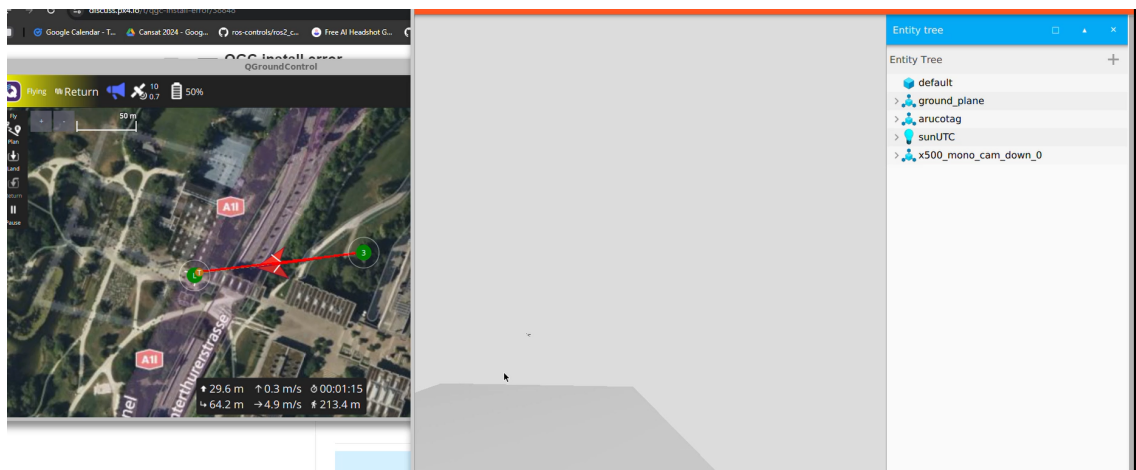


Figure 4.6: Proceeding towards drop zone

As the drone approaches its home location, it performs a critical validation step before beginning its descent. It actively scans the ground using its downward-facing camera to

detect the presence of an ArUco marker at the landing site. This marker serves as a fiducial reference, providing precise positional and orientation data for the drone.

Once the ArUco marker is detected and its location confirmed, the drone aligns itself with the marker to ensure an accurate and safe descent. This final phase involves continuous visual feedback from the camera, allowing the drone to make real-time adjustments to its trajectory. Only after confirming alignment with the ArUco marker does the drone begin its descent, landing precisely at the designated spot. This combination of GPS navigation and marker-based landing ensures both broad navigational accuracy and precise local positioning, critical for successful autonomous operations.

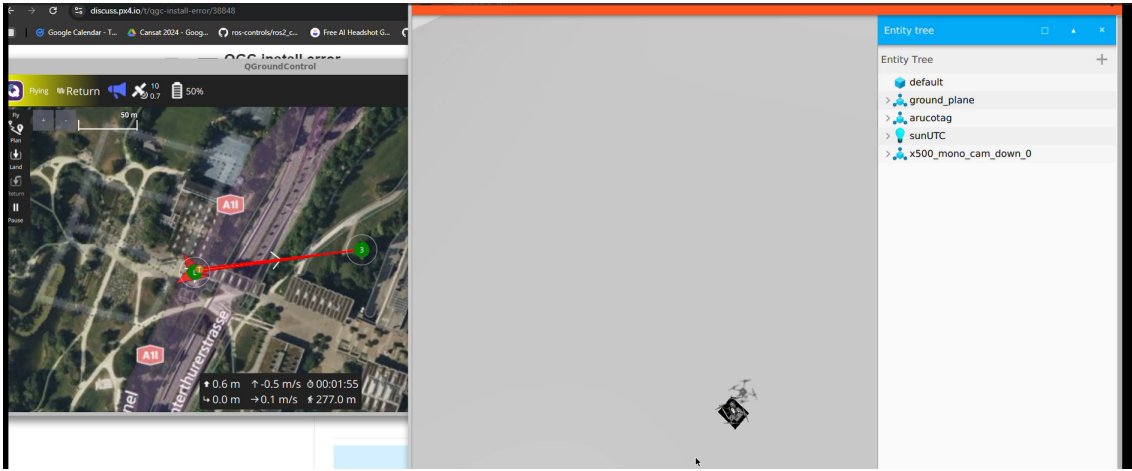


Figure 4.7: Landing back at the Aruco Marker

4.2 Results

The ROS2 simulation flight closely mirrored what would occur in a real-world scenario, providing an accurate representation of the drone's behavior during its autonomous operations. The simulation allowed for the fine-tuning of various aspects of the system, from navigation algorithms to payload delivery, ensuring that the drone performed its tasks efficiently and accurately within a controlled virtual environment. By using this simulation, potential issues could be identified and addressed before moving to physical testing, minimizing risks and improving the system's overall reliability.

However, while the simulation offers a valuable tool for testing, it is crucial to consider real-world factors that might affect drone performance. Further testing under various environmental conditions, such as wind, temperature variations, and other weather-related challenges, is necessary to better understand how the drone adapts to these external influences. For example, wind resistance can affect the drone's stability and navigation accuracy, while temperature changes may impact battery performance and sensor cali-

bration. By testing the drone in a range of real-world conditions, the team can evaluate its resilience, refine its performance, and ensure that the system can handle unpredictable factors during actual operations.

These additional tests will provide further insights into the robustness of the hardware and software, ensuring that the autonomous payload delivery system is capable of functioning efficiently in dynamic, real-world environments. This next phase of testing will be vital for validating the system's readiness for deployment in operational scenarios, where environmental conditions are always a factor.

Chapter 5

Hardware Testing

Preliminary hardware testing began with manually flying the drone to evaluate its stability and performance under controlled conditions. This phase was crucial for identifying any inherent design or mechanical issues that could compromise flight dynamics. The drone was manually controlled to perform a range of basic maneuvers, including hovering, forward flight, and directional changes, to assess its balance, responsiveness, and ability to maintain stable flight.

During these tests, particular attention was paid to how the drone handled external factors such as wind resistance and vibration. Observations were made regarding the behavior of the frame, motors, and propellers, ensuring that no excessive oscillations or imbalances affected stability.

Additionally, the testing allowed for the identification of potential issues in the hardware setup, such as misaligned propellers, imbalanced weight distribution, or inadequate motor thrust. These factors, if left unaddressed, could result in erratic flight behavior or increased strain on the drone's components, reducing overall efficiency and reliability.

By conducting this initial stability check in a manual flight mode, the team was able to validate the fundamental hardware design and make adjustments as necessary. The insights gained from this testing phase formed a baseline for further development and fine-tuning of the system, ensuring the drone was stable and reliable before integrating autonomous features such as precision landing or payload delivery mechanisms.



Figure 5.1: Drone built for autonomous delivery

Chapter 6

Conclusion

The simulation for the autonomous payload delivery system begins with a command from QGroundControl (QGC) to arm the drone and initiate takeoff. After gaining stability, the drone ascends to a pre-defined altitude and autonomously navigates to the specified GPS coordinates for the payload drop. Once at the target location, the drone hovers to release the payload with precision. After completing the delivery, it receives a return-to-home (RTH) command and retraces its path back to the launch point. On arrival, the drone does not immediately descend but instead scans for the presence of an ArUco marker using its downward-facing camera. Once the marker is confirmed, the drone aligns itself and begins a controlled descent, ensuring a safe and precise landing.

This project represents a milestone in the development of an autonomous payload delivery system. The journey began with a structured approach that defined clear objectives and specifications, followed by CAD modeling to create robust hardware capable of integrating navigation algorithms, control systems, and sensors for autonomous operation. The result was the creation of a functional prototype.

Initial tests using the cost-effective SpeedyBee controller were conducted to mitigate risks associated with expensive components like the Pixhawk flight controller. These tests highlighted critical challenges, such as instability during flight and crashes caused by GPS inaccuracies and high G-forces. These insights underscored the need for stabilization mechanisms, prompting plans to integrate a vibration-reducing mount to enhance flight performance.

Software and simulation were integral to the project's success. ROS2 provided a virtual environment to test and refine navigation and control algorithms, reducing risks during physical trials. Image processing capabilities, specifically for detecting ArUco markers, added precision to payload delivery, with Raspberry Pi code adapted to handle marker localization and identification. While full integration of image processing remains

a future task, the groundwork laid is promising for improving system accuracy.

Iterative testing and refinement combined simulation with real-world trials, identifying weaknesses and enabling systematic improvements. Insights gained from SpeedyBee testing emphasized the importance of GPS reliability and stability controls for successful operation.

This project is a testament to the feasibility of using autonomous drones for payload delivery, marking the successful completion of a functional prototype. Beyond this milestone, the work serves as a foundation for future enhancements, including better image processing, advanced stabilization, and improved navigation. These developments promise to evolve the system into a robust, efficient, and versatile solution for real-world applications.

Chapter 7

Future Scope

To enhance the reliability and performance of the autonomous payload delivery system, the following steps will be prioritized in upcoming phases:

1. **Field Tests with Pixhawk Integration:** Further testing will be conducted using the Pixhawk flight controller, focusing on parameter optimization to address the stability issues and GPS navigation challenges identified during prototype testing.
2. **Development of a Stabilization Mount:** A dedicated stabilization mount will be designed and integrated to minimize vibrations and manage high G-forces more effectively, ensuring smoother and safer drone operation.
3. **Integration of Image Processing with ArUco Markers:** The Raspberry Pi-based image processing module will be integrated into the system to enhance localization and improve precision in payload delivery. The utilization of ArUco markers will facilitate reliable object detection and positioning.
4. **Expanding Testing Environment:** Field trials will be extended to include more diverse and challenging conditions to test the drone's performance and robustness under varying operational scenarios.
5. **Obstacle Avoidance:** The obstacle avoidance technologies will be incorporated to ensure safe and efficient operation in dynamic environments.

By addressing these areas, the system will evolve into a more stable, efficient, and versatile solution, capable of meeting both technical and operational.

Bibliography

- [1] Brunner, Gino, Bence Szebedy, Simon Tanner, and Roger Wattenhofer. “The Urban Last Mile Problem: Autonomous Drone Delivery to Your Balcony.”
- [2] Deshpande, Ishwari, Abhishek Patil, and Vinaya Tapkir. “Aerial Delivery System.” 2024.
- [3] Qays, Hasan Mudhafar, Salman, Aymen Dawood, and Jum, Bassim Abdul Baki. “Autonomous Quadcopter Delivery System.” *Journal of Xi'an University of Architecture & Technology*, Vol. XII, Issue IV, 2020.
- [4] Kalta, Shreya, Chauhan, Harsh, Bawa, Guladab, Bag, Abhisek, and Singh, Gunmeet. “Developing a Prototype for Autonomous Drones.”
- [5] Tauseef, Md., M. P., Thanushree, Sravani, K., R., Lohith, and Reddy, T. Sai Sankalp. “Secure Skies: Developing a Remotely Piloted Aerial System with Defense Mechanisms.”
- [6] “Image Processing of Aruco Markers using Raspberry Pi.” Github. <https://github.com/>.
- [7] ”ROS2 PX4 Precision Landing with ArUco Markers” Github <https://github.com/ARK-Electronics/tracktor-beam/tree/main>